| Awareness <br> learning about the problem | Consideration <br> looking for solution ideas | Decision <br> is this the right solution|
| ------ | -------- |-------- |
| topic page? | solution page | proof points |
| landing pages? | ?comparisons? | comparisons |
| -etc? | | - product page x <br> - product page y <br> - product page z |

---
## SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/agile/keywords.md

SEO and Keywords for Agile

- Keyword
- etc.

---
## SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/agile/message-house-template.md

| Positioning Statement: | (how GitLab fits and is differentiated in the market for this use case) |
|---|---|
| User Persona | a few words and a link to the details |
| Buyer Persona | a short description and a link to details |
| Short Description | 25 words or less |
| Long Description | 100 words or less |

| Key-Values | Value 1: <List a key message/ value proposition> | Value 2: | Value 3: |
|---|---|---|---|
| Promise | (list and describe the positive business outcomes) | | |
| Pain points | (describe common pain points) | | |
| Why GitLab | (list specific features that support this value) | | |

| Proof points | (list specific analyst reports, case studies, testimonials, etc) |

---
## SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/cd/_index.md

Who to contact

| Product Marketing | Developer Advocate |
| ---- | --- |

| Daniel Hom (@danielhom) | Cesar Saavedra ( @csaavedra1 ) |

## The Market Viewpoint

### Continuous Delivery

> "Deployment is manual"
> "Functional tests are manual"
> "Time consuming or lack of rollback on performance degradation or production errors"
> "Hard to maintain environment configurations and hard to operate"
> "No consistency in deployment process"
> "Manual / hard coded configurations"
> "No standardized software artifact"
> "No release management in place"
> "Too dependent on other teams to get any release done"

If these are the typical problems you face, Continuous Delivery is for you.

Continuous Delivery is the next logical step after continuous integration and it streamlines and automates the application release process to make software delivery repeatable and on demand - from provisioning the infrastructure environment to deploying the tested application software to test/staging or production environments. Organizations practicing continuous delivery are able to plan their release processes and schedules, automate infrastructure and application deployments, manage deployed infrastructure and application resources resources, and analyze metrics to optimize the software delivery process.

### Why Continuous Delivery?

- Consistent & repeatable release process - lesser manual processes imply the release process is less error prone and hence can be repeatable for every minimal change to the code
- Faster time to market - automation of environment provisioning, software deployment and rapid feedback helps teams to iterate faster and rollback when necessary
- Lower risk releases - by using progressive delivery practices such as advanced deployments: incremental / blue green / canary deployments, review apps, feature flags and a deployment performance feedback loop, organizations are able to validate their software before widespread deployment

## Personas

### User Persona

The typical user personas for this use case are:

### DevOps Engineer, Devon

The DevOps engineer is the stable counterpart for the Developer to aid with support of the infrastructure, environment and integrations necessary for the developer to deploy their code to test/staging or production environments.

### Systems Administrator, Sidney

The Systems administrator is the infrastructure expert - who contributes to modeling, maintaining and scaling the test/staging and production environments - including physical, virtual or cloud infrastructure and the application infrastructure like databases and middleware.

Release Manager, Rachel

The release manager has a central role in release planning, scheduling, identifying dependencies and resources to ensure that the release is timely. The release manager helps automating the release process.

Platform Engineer, Priyanka

The platform engineer is a specialist in modern platforms and aims to empower developers to provision, deploy and decommission tiered environments in a self service manner.

Application Operations, Allison

The operations specialist ensures that the deployed application is available and performing to the required performance parameters.

Buyer Personas

The typical buyer personas for this use case are:

Infrastructure Engineering Director, Kennedy

The Infrastructure Engineering Director is responsible for building and scaling highly available environments. He/She frequently has the agenda of Cloud initiatives and Cost Optimization in the organization.

Release and Change Management Director, Casey

The Release and Change Management Director is responsible for managing complex releases from concept to delivery. The CIO may be the final decision maker or buyer, but the Release and Change Management Director has significant influence in the buying process.

Industry Analyst Resources

Examples of comparative research for this use case are listed just below. Additional research relevant to this use case can be found in the Analyst Reports - Use Cases spreadsheet.

Market Requirements

| Market Requirement | Description | Typical capability-enabling features | Value/ROI |
|---|---|---|---|
| 1) Release Planning | The solution should be able to define the planning of the release workflow which includes determining what goes into the release (Bill of Material of applications & services), what are the dependencies (application / micro services dependencies), who will do it (people resource management), when will it be done (scheduling),

what is the readiness criteria, who will approve the release | - Bill of materials (release modeling) <br/> - Release dependencies <br/> - Release Versioning <br/> - Sequence of the release <br/> - Schedule of events and release calendar <br/> - Resource planning including forecasting <br/> - Readiness criteria <br/> - Approval gates <br/> - List of isuues in the release and their status <br/> - Release Evidence | |
| 2) Manage the artifacts and binary assets | The solution should be able to manage the inputs from continuous integration i.e., artifacts and binary assets to deploy the artifacts to the test, staging or production environments. | - Maintain versions, dependencies, meta data for the application <br/> - Maintain container images <br/> - Retrieve application / binary artifacts for deployment <br/> - Separation of duties and access control <br/> - Support a range of common package formats and third party integrations <br/> - Repository / registry can be used on-prem or in the cloud | |
| 3) Environments management (i.e., Operating Environment) | The solution should be able to enable consistent and repeatable modeling of the environment for test, staging and production - including on-prem, virtual, cloud (a mix of multi and hybrid cloud environments), maintain a system of record of the environment & various elements of the environment (akin to a CMDB) | - Infrastructure modeling (via UI / Infrastructure as a code, blueprints, runbooks) <br/> - Support hybrid infrastructure environments in modeling (phyical, virtual, cloud (both multi & hybrid)) <br/> - System of record of various environments (test, stage, production) <br/> - System of record of configurations & policies <br/> - Access control / approvers for environment changes <br/> - Configuration & Policy Change Management <br/> - Automated environment discovery | |
| 4) Database Provisioning | The solution should be able to model, provision and deploy to databases required to support the running application | - Model database dependencies <br/> - Discovery of databases <br/> - Provisioning and configuration of databases such as schema, stored procedures <br/> - Loading data (data provisioning) <br/> - Access control / approvers <br/> - Configuration Change | |
| 5) Middleware Provisioning | The solution should be able to model, provision and deploy to middleware software required to support the running application | - Model middleware dependencies <br/> - Discovery of middleware <br/> - Configuration of middleware servers & clusters <br/> - Access control / approvers <br/> - Configuration Change | |
| 6) Application Release Automation & Delivery | The application should be able to automate the end to end release activities including build & test (which is covered as part of continuous integration) and deployment automation which includes scheduling various tasks, deploying the application to the desired environments, rollout scenarios, rollback and system validation | - Delivery Pipelines <br/> - Pipeline versioning <br/> - Task Scheduling & Sequencing <br/> - Rollout scenarios such as canary, incremental roll out, blue green deployments <br/> - Feature Flags <br/> - Review Apps <br/> - Performance testing & validation | |
| 7) Resource allocation and management | The application should be able to provide a detailed and summarized view of the costs associated with the infrastructure and application infrastructure modeled as well as optimization recommendations | - Cost management <br/> - Cost optimization | |
| 8) Multi Platform/Cloud/Integration Support | The application should be able to play well with multiple clouds, multiple platforms (e.g., Linux, Unix, Windows, container platforms, mainframe, midrange, mobile, specialized), multiple integrations (e.g., CMPs, Registries, Orchestration tools, APM tools, etc) | - Cloud Support (AWS, GCP, Azure, IBM, Oracle, etc) <br/> - Platform Support (Linux, Unix, Windows, container platforms, mainframe, midrange, mobile, specialized) <br/> - Integration Support (CMPs, Registries, Orchestration tools, APM tools) | |
| 9) Governance and Compliance | The solution should be able to enforce separation of duties,

access control, maintain a system of record of changes for compliance purposes, maintain release traceability back to requirements, enforce information security checks and policies | - Separation of duties including role based access control to pipelines and deployment environment <br/> - Credential management <br/> - Approver gates <br/> - Traceability to requirements <br/> - Security checks <br/> - Change logs <br/> - Compliance reports |  |
| 10) Analytics and reporting | The solution should be able to provide analytics and reports to visualize release status & statistics, pipeline status & statistics, deployment status & statistics, environment status & statistics, change reports for compliance | - Release status & statistics like release plan, timeline, status <br/> - Pipeline status & statistics like success, failure rates, pipeline heath <br/> - Deployment status & statistics like deployment frequency, change failure rates (DORA metrics) <br/> - Environment status & statistics like usage, availability, downtime, failure rates <br/> - Change logs, approvers & compliance reports - Release Evidence <br/>  |  |
| 11) Enterprise readiness | The solution should be able to support enterprise capabilities such as High Availability / Disaster Recovery, secure storage of data, access control | - High Availability, Disaster Recovery <br/> - Secure data storage <br/> - Separation of duties and access control |  |

The GitLab Solution

<iframe width="960" height="569" src="https://www.youtube.com/embed/QArt7rqfbqk" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

How GitLab Meets the Market Requirements

A collection of short demonstrations that show GitLab's CD capabilities.

| Market Requirements | How GitLab Delivers | GitLab Stage/Category | Demos |
| ------ | ------ | ------ | ------ |
| 1) Release Planning | A GitLab Release is a snapshot of the source, build output, artifacts, and other metadata associated with a released version of your code. The release evidence contains the bill of material of the release i.e., everything in the release including release milestones and release assets | Release Stage: Release Orchestration, Release Evidence | tbd |
| 2) Manage the artifacts and binary assets  | In the CD usecase, artifacts already created in the CI usecase can be viewed, downloaded, edited and shared. Various formats including maven, npm, nuget, amongst others are supported | Package stage: Package Registry, Container Registry, Dependency Proxy | tbd |
| 3) Environments management (i.e., Operating Environment) | GitLab leverages partners like Terraform to model and discover hybrid environments. GitLab supports storing these environments and configurations as code, maintaining a system of record of various environments and their configurations as code, snapshot view of the environments in a dashboard and deploying to hybrid infrastructure environments. While you can use GitLab CD to deploy apps almost anywhere, GitLab naturally supports Kubernetes, with a keen sight to improving non-cloud native support | Configure stage: Auto DevOps, Kubernetes Management, Runbooks, Infrastructure as Code, Environments Dashboard <br> Terraform based infrastructure automation| tbd |
| 4) Database Provisioning | GitLab integrates with Terraform to enable model and provision infrastructure, including databases. GitLab enables Infrastructure as code for Terraform - maintaining infrastructure and configurations of environments in source control within GitLab | Terraform based infrastructure automation, Infrastructure as code with Terraform and GitLab |

tbd |
| 5) Middleware Provisioning | GitLab integrates with Terraform to enable model and provision infrastructure, including middleware. GitLab enables Infrastructure as code for Terraform - maintaining infrastructure and configurations of environments in source control within GitLab | Terraform based infrastructure automation, Infrastructure as code with Terraform and GitLab | tbd |
| 6) Application Release Automation & Delivery | GitLab supports multiple advanced deployment strategies including progressive and incremental delivery. Review apps provide an opportunity to preview web applications before deployment, feature flags allow you to control the audience of features. GitLab CI/CD pipelines can be architected to configure and sequence your pipeline, gitlab-ci.yml file can be used to setup and define pipeline versions. Additionally, perform post deployment monitoring using browser performance testing for web applications and application performance testing using the monitor stage capabilities | Release Stage: Continuous Delivery, Review Apps, Advanced Deployments, Feature Flags, Release Evidence, Secrets Management <br> Monitor Stage: Metrics, Logging, Tracing, Error Tracking <br> Verify: Browser Performance Testing | Application Release Automation & Delivery <br> Feature flags |
| 7) Resource allocation and management | Users can utilize GitLab CI and Monitoring capabilities to chart their resource allocation and consumption and setup alerts when thresholds have been met as well as view cost implications of proposed Infrastructure as Code changes in their Merge Request. Native support for this capability is part of the GitLab roadmap | Cluster Cost Optimization Limiting the number of deployments to a specific resource - If multiple jobs belonging to the same resource group are enqueued simultaneously, only one of the jobs is picked by the runner, and the other jobs wait until the resourcegroup is free. | tbd |
| 8) Multi Platform/Cloud/Integration Support | GitLab can be installed on AWS, Google Cloud, Azure and can be deployed to multiple clouds including AWS, Google Cloud, Azure, VMWare, IBM amongst others. GitLab installations support only Linux based distributions. | All Stages: GitLab Installation Clouds Cloud Deployment Targets, Install Requirements, Integrations | tbd |
| 9) Governance and Compliance | Compliance testing and audit controls are built into GitLab's CI pipelines. | Compliance at GitLab <br> Manage Stage: Audit Events, Audit Logs, Audit Reports, Compliance Management, Release Evidence <br> Secure Stage: License Compliance, Dependency Scanning| tbd |
| 10) Analytics and reporting | GitLab provides a variety of Executive Insights, Productivity Insights, Operations Insights and Security Insights | All Stages: <br> Executive Insights DevOps Score, Value Stream Analytics, CI/CD Charts, Roadmaps <br> Operations Insights: Operations Dashboard, Environments Dashboard, Environments <br> Other insights such as Productivity Insights and Developer Insights are applicable to other usecases| tbd |
| 11) Enterprise readiness | GitLab supports enterprise grade authentication and authorization, access management, audit information, compliance, high availability and disaster recovery, geographic replication for great user experience across locations, large user reference architectures, infrastructure as code amongst others | All Stages particularly Manage Stage, Enablement Section | tbd |

Top Roadmap Items for CD

- Natively support hypercloud deployments
- Advanced deploys (Blue/green, Canary, Traffic vectoring)
- Streamline AWS Deployments
- Advanced Deployments for AWS
- Get Feature Flags to Enterprise Grade

- Feature Flag Strategies
- Post-deployment monitoring (continuous verification) MVC
- A/B testing based on Feature Flags
- Feature Flags with Review Apps enabled
- Pre-packaged and extensible deploy templates
- Review Apps For Deployment to Mobile

Top 3 GitLab Differentiators

| Differentiator | Value | Proof Point | Demos |
|---|---|---|---|
| 1) Unified deployment and monitoring strategies | GitLab provides the ability to visualise what goes into production (via Review Apps), what to deploy to production (via Feature Flags), who to deploy it to (via Progressive Delivery and deployment strategies like Canary), monitor performance of deployment (via browser performance testing, performance monitoring/tracing) and rollback based on performance via post deployment monitoring, all from a single application. | Strong Performer in the Forrester Wave for Continuous Delivery and Release Automation Q2 2020 <br/> - James Governor from RedMonk talking about GitLab's focus on Progressive Delivery - GitLab is an interesting case for a couple of key reasons: <br/> a) It has a significant and fast growing enterprise footprint, notably with its CD product. <br/> b) It develops software at high velocity, with a monthly release schedule and a strong bias to shipping, so you can watch improvements in real time. The company ships open source software and has a strong culture of corporate observability. | Unified and integrated monitoring and deployment strategies |
| 2) Automated and Integrated Continuous Delivery | GitLab Auto DevOps simplifies and accelerates delivery with a complete delivery pipeline out of the box. Simply commit code and GitLab does the rest. GitLab also provides an integrated dashboard that spans across the CI/CD pipeline status and deployment status | - The built-in features of Auto DevOps have made our experience more rewarding and effective - Daniel B on G2 Peer Reviews<br/> - It has really helped us to shorten lead time, which has positively affected every single metric we measure - Chorus.ai <br/> - GitLab Auto DevOps also delivered the technology component required for true CI/CD, accelerating product delivery with an end-to-end pipeline out of the box. - ExtraHop Networks | Automated and integrated Continuous Delivery |
| 3) Modern Compliance for Continuous Delivery | GitLab simplifies compliance with helping customers define granular policies such as who can approve MR, push to production, segregation of duties, release governance etc, define security policies such as license compliance, password policies, credential inventories etc, track adherence to compliance such as user actions such as commits, permission changes, approval changes, logins, password changes, release evidence etc - all within a single application which allows traceability from deployment all the way back to code changes and requirements | - During a recent audit for SOC2 compliance, the auditors said that Chorus had the fastest auditing process they have seen and most of that is due to the capabilities of GitLab - Chorus.ai <br/> - There is no longer a need for license keys or several different logins, because of the built-in security and compliance. Software is deployed anywhere, which relieves developers localization constraints.| tbd |

Message house

The message house provides a structure to describe and discuss the value and differentiators for Continuous Delivery with GitLab.

Customer Facing Slides

<figure class="videocontainer">
<iframe src="https://docs.google.com/presentation/d/1bGdjQNfHxmYKYzZsrtyhEyXLGlv8UoTaviaGl3UNc/embed?start=false&loop=false&delayms=3000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>
</figure>

Discovery Questions

-

Sample Discovery Questions

-

Additional Discovery Questions

-

Industry Analyst Relations (IAR) Plan

- The IAR Handbook page has been updated to reflect our plans for incorporating Use Cases into our analyst conversations.
- For  details specific to each use case, and in respect of our contractual confidentiality agreements with Industry Analyst firms, our engagement plans are available to GitLab team members in the following protected document: IAR Use Case Profile and Engagement Plan.

For a list of analysts with a current understanding of GitLab's capabilities for this use case, please reach out to Analyst Relations via Slack (analyst-relations) or by submitting an issue and selecting the "AR-Analyst-Validation" template.

Competitive Comparison

Proof Points - Customer Recognitions

Quotes and reviews

Gartner Peer Insights

Gartner Peer Insights reviews constitute the subjective opinions of individual end users based on their own experiences, and do not represent the views of Gartner or its affiliates. Reviews have been edited to account for errors and readability.

"GitLab is the most preferred service in the world and its user community is very wide. We can authorize project or branch based user authorization on GitLab. In addition, continuous deployment integrations can be done very quickly. In addition, you can create merge requests within the constraints you want and easily manage them. It is very easy to prevent conflicts. A service that must be used for software development teams."

> - Software Development Lead, Gartner Peer Insights Review

"GitLab supports my [company's] entire continuous integration and continous delivery(Ci/Cd) process. It has a smooth integration with Jira , which we use for software process management. "

> - Principal Android Engineer, Gartner Peer Insights Review

"At my company we use [GitLab] to host all the different projects as it very easy to use and collaborate with may developers. Every project has access to specific set of people who has access to view, build features in the project. Peer reviews are very simple to view the code changes in a split window. Easy to create pipelines with CI/CD"

> - Software Engineer, Gartner Peer Insights Review

G2

"For me, the most impressive part of their toolchain would have to be the CI/CD Platform, the ease of use and it's flexibility is wonderful. Building CI/CD pipelines never felt easier."

> - Luca Favaretto Marques, Software Engineer, Mid-Market, G2

"GitLab does a great job at creating a unified experience for our developers. We previously had several best-of bread solutions (code repository, issue tracker, CI runners and deployment pipelines) co-exist between our team, but we managed to consolidate this into a single solutions, which meets most of our needs."

> - Joël Cox, Partner, Small Business, G2

"The main reason why I chose GitLab over years of using Github was because of their CI/CD tool. Github doesn't come with it out of the box and we needed a solution in a team where nobody is a DevOps but js developers"

> - Cynthia Sanchez, Founder, Product-Manager, SMB, G2

"GitLab has helped me master Git, CI/CD pipelines, and software development in general through the many resources it offers. I don't have to spend time learning so many separate services and figuring out how everything fits together. I would strongly recommend it to anyone looking for a great tool for their development activities."

> - Justin Smith, System Administrator, Mid-Market, G2

Garter Peer Insights 'Voice of the Customer'

GitLab Recognized as a Gartner Peer Insights Customers' Choice for ARO

> - Gartner Peer Insights 'Voice of the Customer' Application Release Orchestration 2020

Blogs

## Wag

Wag!

- Problem: Slow, fragile and manual release process impacted developer efficiency
- Solution: GitLab Premium (CI/CD)
- Result: What previously took 40 minutes to an hour to accomplish, now takes just six minutes.
- Sales Segment: SMB
- Safe Deployments (https://gitlab.com/gitlab-com/www-gitlab-com/-/mergerequests/51235)

## Athlinksx

Athlinks

- Problem: Complex toolchain that hindered deploy times and disempowered developers
- Solution: GitLab Ultimate (SCM,CI,CD) and Terraform
- Result: Athlinks cuts runtime in half with GitLab
- Sales Segment: Enterprise

## Case Studies

### Hemmersbach

- Problem Hemmersbach was burdened by multiple tools and communication inefficiencies, resulting in slow production builds and manual processes
- Solution: GitLab Ultimate (CI/CD)
- Result: Having all of the collaboration capabilities under one umbrella has enabled unprecedented deployment speed (up to 30 automated daily deploys)
- Sales Segment: Enterprise

### BI Worldwide

- Problem BI Worldwide was looking for a way to increase collaboration and efficiency in its developer environment and to reduce toolchain complexity
- Solution: GitLab Ultimate (SCM/CI/CD)
- Result: Deployments increased to 10 times daily
- Sales Segment: Enterprise

### Glympse

Glympse

- Problem A complex developer tech stack with over 20 distinct tools that was hard to maintain and impeded innovation
- Solution: GitLab Ultimate (SCM/CI/CD)
- Result: 8 times faster deploys (from 4 hours to less than 30 minutes)
- Sales Segment: Enterprise

### KnowBe4

- Problem KnowBe4 was looking for a tool to keep code in-house and that offered the capabilities of several tools in one
- Solution: GitLab Ultimate (CI/CD) and AWS
- Result: 5+ production deploys per day for any given application plus 20+ development environment deploys per day
- Sales Segment: Enterprise

MGA

- Problem MGA was looking for a cost efficient CI platform that could improve workflow, knowledge, and code quality.
- Solution: GitLab Starter (SCM/CI/CD)
- Result: 10 times better success rate with CD than with manual deploys plus 80% time saved moving to CD
- Sales Segment: SMB

References to help you close

SFDC Report of referencable Release customers.  Note: Sales team members should have access to this report. If you do not have access, reach out to the customer reference team for assistance.

Request reference calls by pressing the "Find Reference Accounts" button at the top of your stage 3 or later opportunity.

Adoption Guide

The following section provides resources to help CSMs lead capabilities adoption, but can also be used for prospects or customers interested in adopting GitLab stages and categories.

Playbook Steps

1. Ask Discovery Questions to identify customer need
2. Complete the deeper dive discovery sharing demo, proof points, value positioning, etc.
3. Deliver pipeline conversion workshop and user enablement example
4. Agree to adoption roadmap, timeline and change management plans, offering relevant services (as needed) and updating the success plan (as appropriate)
5. Lead the adoption plan with the customer, enabling teams and tracking progress through engagement and/or product analytics data showing use case adoption

Adoption Recommendation

This table shows the recommended use cases to adopt, links to product documentation, the respective subscription tier for the use case, and product analytics metrics.

| Feature / Use Case | F | P | U | Product Analytics | Notes |
| ---------- | -----| ---- | ---- | --------- | ---- |
| Try Auto DevOps | x | x |x | instanceautodevopsenabled and counts.cipipelineconfigautodevops

| |
| Setup GitLab CI | x | x | x | | Having a .gitlab-ci.yml is the basis of using GitLab for deployment |
| Setup an Environment | x | x | x | counts.environmnets | |
| Deploy to an Environment | x | x | x | counts.deployments, usageactivitybystagemonthly.release.deployments | |
| Create a Release | x | x | x | counts.releases | |
| Create Release Evidence | x | x | x | | |
| Setup and use feature flags | x | x | x | | |

Additional Documentation Links

- Introduction to CI/CD with GitLab
- Getting started with GitLab CI/CD
- GitLab CI/CD Examples

Enablement and Training

The following will link to enablement and training videos and content.

- Make Your Life Easier with CI/CD Presentation
- CI/CD Overview Video
- CS Skills Exchange: CI Deep Dive
- CS Skills Exchange: Runners Overview
- CS Skills Exchange: Runners Overview
- Technically Competing Against Microsoft Azure DevOps (GitLab internal only)
- Competing Against Jenkins (GitLab internal only)
- Coming soon.... CD Learning Path

Professional Service Offers

Key Value (at tiers)

Core/Free

Why choose GitLab Core/Free for CD?
We are committed to lowering the barriers for organizations embarking on their CI/CD journey. In March 2020, we announced a number of features CD features that are moving to core.

Key features with Core/Free:

- Package repository: private repository for a variety of package managers
- Deployment Strategies: support for canary deployments, incremental roll outs, blue green deployments and feature flags to give you confidence in your releases
- Deploy boards: gives a consolidated view of health and status of Kubernetes deployments
- Multiple Kubernetes Clusters: allows you to maintain different clusters for different environments such as for test, staging and production
- Environments and Deployments: configure multiple environments, manage and monitor them from GitLab
- GitLab Pages: deploy static web pages directly from GitLab

- Deploy Tokens: secure your package and container registry images by requiring username/password for access
- Release Evidence: snapshot of releases data to compare and audit releases
- Vault integrations: authentication of secrets via Hashicorp Vault
- ChatOps: interact with GitLab via chat services
- AutoDevOps: simplify build, test, deploy, monitor of your applications

Premium

Why choose GitLab Premium for CD?
Premium is ideal for scaling organizations for multi team usage, enabling organizations scale their DevOps delivery with advanced configuration, consistent standards and compliance. Take advantage of enterprise level priority support, including 24/7 uptime support, a named Customer Success Manager (CSM), and upgrade assistance.

Key features with Premium:

- Dependency Proxy - local proxy for packages
- Multi Project Pipelines- link CI pipelines from multiple projects.
- Operations dashboard- get a holistic view of the overall health of CI/CD pipelines and organization wide operations.
- Environments dashboard - cross project environment based view to track deployment status
- CI/CD for external repositories- connect your external repositories instead of moving your entire existing project(s) to get the benefits of GitLab CI/CD. This feature supports GitHub, Bitbucket Cloud, and any other Git-based repository.

Ultimate

Why choose GitLab Ultimate for CD?
Ultimate is ideal for projects with executive visibility while managing priorities, security, risk, and compliance.

Key features with Ultimate:

- Compliance dashboard - high level view of project compliance status and merge request approvers
- Container Scanning- analyze Docker images and check for potential security issues.
- Dynamic Application Security Testing- analyze review applications to identify potential security issues on running web applications before deployment

Resources

What is CI/CD?

Check out this introductory video to learn the basics of CI/CD as software development best practices and how they apply with GitLab CI/CD!
<!-- blank line -->
<figure class="videocontainer">
<iframe src="https://www.youtube.com/embed/nLwJtVWXN70" frameborder="0" allowfullscreen="true"> </iframe>

</figure>
<!-- blank line -->

Presentations

- Why CI/CD?

Continuous Delivery Videos

- CI/CD with GitLab
- GitLab for complex CI/CD: Robust, visible pipelines
- How do Runners work?
- Mastering CI/CD
- What is Auto DevOps?

Integrations Demo Videos

- Migrating from Jenkins to GitLab
- Using GitLab CI/CD pipelines with GitHub repositories

Clickthrough & Live Demos

- Live Demo: GitLab CI/CD Deep Dive

Blogs and articles

- Auto DevOps 101: How we're making CI/CD easier
- Progressive Delivery

Interesting reads

- How We Switched to a Continuous Delivery Pipeline in 3 months

Buyer's Journey

Inventory of key pages in the buyer's Journey

| Awareness <br> learning about the problem | Consideration <br> looking for solution ideas | Decision <br> is this the right solution|
| ------ | -------- |-------- |
| topic page? | solution page | proof points |
| landing pages? | ?comparisons? | comparisons |
| -etc? | | - product page x <br> - product page y <br> - product page z |

---
## SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/cd/message-house.md

Messaging and Positioning

| Positioning Statement | Continuous Delivery with GitLab automates application delivery for DevOps engineers, Operations and Release Management Personnel via an out of the box, integrated solution to create application delivery pipelines that are consistent, repeatable and compliant. |
|---|---|
| Short Description | GitLab's Continuous Delivery helps customers automate application deployment with repeatable and compliant delivery processes from a single application. |
| Long Description | As developers build code faster and organizations move towards more frequent deployments, automating the application delivery processes is crucial. Automation gives organizations better control over the deployments (what is delivered, when it is delivered, what features, who can approve etc) and allows teams to revert back to a previous state if needed. Without adequate controls, applications can become out of sync. Without adequate automation, organizations are forced to manually cobble up different tools to create a coherent Continuous Delivery pipeline. |

| Key-Values | CONSISTENT | EFFICIENT | COMPLIANT |
|---|---|---|---|
| Promise | - Build repeatable application delivery processes | - Integrated delivery pipeline to minimize manual cobbling together of tools <br> - End-to-end visibility of delivery pipeline and status | - Frictionless compliance with adequate controls to manage the application delivery process |
| Pain points | - Deployment is too manual <br> - Hard to maintain consistency in environments and configurations <br> - Hard to detect performance issues due to deployment and rollback | - Manual/hard-coded processes <br> - Too dependent on other teams <br> - Team spending too much time in creating and maintaining toolset <br> - No unified view of deployment status | - Cannot determine adequately what went into a release, who approved it, was the action authorized etc |
| Why GitLab | GitLab provides the ability to visualize what goes into production (via Review Apps), what to deploy to production (via Feature Flags), who to deploy it to (via Progressive Delivery and deployment strategies like Canary), monitor performance of deployment (via browser performance testing, performance monitoring/tracing) and rollback based on performance - all in a single application | GitLab Auto DevOps simplifies and accelerates delivery with a complete delivery pipeline out of the box. Simply commit code and GitLab does the rest. GitLab also provides an integrated dashboard that spans across the CI/CD pipeline status and deployment status | GitLab simplifies compliance with helping customers define granular policies such as who can approve MR, push to production, segregation of duties, release governance etc, define security policies such as license compliance, password policies, credential inventories etc, track adherence to compliance such as user actions such as commits, permission changes, approval changes, logins, password changes, release evidence etc - all within a single application which allows traceability from deployment all the way back to code changes and requirements |
| Customer Proof points | Now it's so easy to deploy something and roll it back if there's an issue. It's taken the stress and the fear out of deploying into production." · Dave Bullock, Wag! | - The built-in features of Auto DevOps have made our experience more rewarding and effective - Daniel B on G2 Peer Reviews<br/> - It has really helped us to shorten lead time, which has positively affected every single metric we measure - Chorus.ai <br/> - GitLab Auto DevOps also delivered the technology component required for true CI/CD, accelerating product delivery with an end-to-end pipeline out of the box. - ExtraHop Networks | - During a recent audit for SOC2 compliance, the auditors said that Chorus had the fastest auditing process they |

have seen and most of that is due to the capabilities of GitLab - Chorus.ai <br/> - There is no longer a need for license keys or several different logins, because of the built-in security and compliance. Software is deployed anywhere, which relieves developers localization constraints. |

---
## SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/ci/_index.md

Looking for a customer-facing overview of GitLab's Continuous Integration (CI) capabilities? See the CI Solution

The page below is intended to align GitLab sales and marketing efforts with a single source of truth for our go-to-market efforts around DevSecOps.

Who to contact

| Product Marketing | Developer Advocate |
| ---- | --- |
| Daniel Hom (@danielhom) | Itzik Gan-Baruch ( @iganbaruch ) |

The Market Viewpoint

Continuous Integration

The Continuous Integration (CI) use case is a staple of modern software development in the digital age. It's unlikely that you hear the word "DevOps" without a reference to "Continuous Integration and Continuous Delivery" (CI/CD) soon after. In the most basic sense, the CI part of the equation enables development teams to automate building and testing their code.

When practicing CI, teams collaborate on projects by using a shared repository to store, modify and track frequent changes to their codebase. Developers check in, or integrate, their code into the repository multiple times a day and rely on automated tests to run in the background. These automated tests verify the changes by checking for potential bugs and security vulnerabilities, as well as performance and code quality degradations. Running tests as early in the software development lifecycle as possible is advantageous in order to detect problems before they intensify.

CI makes software development easier, faster, and less risky for developers. By automating builds and tests, developers can make smaller changes and commit them with confidence. They get earlier feedback on their code in order to iterate and improve it quickly increasing the overall pace of innovation. Studies done by DevOps Research and Assessment (DORA) have shown that robust DevOps practices lead to improved business outcomes. All of these "DORA 4" metrics can be improved by using CI:

- Lead time: Early feedback and build/test automation help decrease the time it takes to go from code committed to code successfully running in production.
- Deployment frequency: Automated build and test is a pre-requisite to automated deploy.
- Time to restore service: Automated pipelines enable fixes to be deployed to production faster

reducing Mean Time to Resolution (MTTR)
- Change failure rate: Early automated testing greatly reduced the number of defects that make their way out to production.

GitLab CI/CD comes built-in to GitLab's complete DevOps platform delivered as a single application. There's no need to cobble together multiple tools and users get a seamless experience out-of-the-box.

## Personas

### User Persona

The typical user personas for this usecase are the Developer, Development team lead, and DevOps engineer.

#### Software Developer Sacha

Software developers have expertise in all sorts of development tools and programming languages, making them an extremely valuable resource in the DevOps space. They take advantage of source code management and CI capabilities together to work fast and consistently deliver high quality code.

- Developers are problem solvers, critical thinkers, and love to learn. They work best on planned tasks and want to spend a majority of their time writing code that ends up being delivered to customers in the form of lovable features.

#### Development Team Lead Delaney

Development team leads care about their team's productivity and ability to deliver planned work on time. Leveraging CI helps maximize their team's productivity and minimize disruptions to planned tasks.

- Team Leads need to understand their team's capacity to assign upcoming tasks, as well as help resolve any blockers by assigning to right resources to assist.

#### DevOps Engineer Devon

DevOps Engineers have a deep understanding of their organization's SDLC and provide support for infrastructure, environments, and integrations. CI makes their life easier by providing a single place to run automated tests and verify code changes integrated back into the SCM by development teams.

- DevOps Engineers directly support the development teams and prefer to work proactively instead of reactively. They split their time between coding to implement features and bug fixes, and helping developers build, test, and release code.

### Buyer Personas

CI purchasing typically does not require executive involvement. It is usually acquired and installed via our freemium offering without procurement or IT's approval. This process is

commonly known as shadow IT. When the upgrade is required, the Application Development Manager is the most frequent decision-maker. The influence of the Application Development Director is notable too.

## Industry Analyst Resources

Examples of comparative research for this use case are listed just below. Additional research relevant to this use case can be found in the Analyst Reports - Use Cases spreadsheet.

- Forrester Wave for Cloud-Native CI Tools

## Market Requirements

| Market Requirement | Description | Typical capability-enabling features | Value/ROI |
|---|---|---|---|
| 1) Build automation | Streamlines application development workflow by connecting simple, repeatable, automated tasks into a series of interdependent automatic builds. This includes compiling, linking, packaging, and ensuring that the binary libraries/files output are ready for testing. Ensure software is consistently built without manual intervention, allowing developers to incorporate automated tests into their pipelines and start verifying/validating code changes. Teams have control over where automated jobs run either in the cloud (public/private/hybrid) or using shared infrastructure. | CI/CD pipelines, scalable resources, job orchestration/work distribution, caching, external repository integrations. Automated builds support a wide variety of languages, frameworks, and libraries development teams rely on. | Increase build speeds. Development teams work more efficiently by reducing otherwise manual work. |
| 2) Test automation | Run and manage automated tests and validate changes before they're merged to production. This includes everything from basic to more in-depth tests and extends test automation into areas of functional, system, performance testing, and more. Ensure software is consistently tested to meet both technical and business requirements without manual intervention, enabling developers to get rapid feedback if their code changes introduce potential defects or vulnerabilities. | Ability to run isolated, automated, tests in CI/CD pipelines. Various tests may include but aren't limited to unit testing, code integration testing, regression testing, static code analysis, functional testing, and accessibility testing. | Catch potential errors sooner rather than later before they intensify.|
| 3) Pipeline configuration management | The CI solution makes it easy for developers to automate the build and test workflow, specifically connecting to their source code repository, defining specific actions/tasks in build pipelines, and set parameters for exactly how/when to run jobs, using scripts and/or a GUI to configure pipeline changes. Configurations are easily reproducible and traceability exists to allow for quick comparisons and tracking of changes to environments. | Configurations via web UI or supports config-as-code in a human readable syntax, like YAML. | Maximize development time and improves productivity. Less manual work. |
| 4) Visibility and collaboration | The solution enables development teams to have visibility into pipeline structure, specific job execution, and manage changes and updates to the pipeline jobs. The solution should enable teams to easily collaborate and make code changes, review work, and communicate directly. CI solutions should provide visibility and insights into the state of any build. | Pull requests or merge requests, commit history, automatic notifications/alerts, chatOps, code reviews, preview environments. | Faster feedback and less risk that changes cause builds to break. |
| 5) Platform and language support | Many organizations use a mix of operating systems and code |

stacks within their development teams.  The CI solution must operate on multiple operating systems (Windows, Linux, and Mac). The CI solution must also be language-agnostic supporting automation of build and test for a wide variety of development languages(Java, PHP, Ruby, C, etc.). | Option to use native packages, installers, downloads from source, and containers. Cloud-native platform support for public, private, or hybrid hosting. Runs on any OS. | Gives teams more flexibility, making it easier to adopt. |
| 6) Pipeline security |  Building security into an automated process ensures access to CI pipelines is consistently managed and controlled. Facilitate secure access to jobs/branches for the right users, safely store/manage sensitive data (such as secrets), and enforce governance and compliance policies without slowing anyone down. | Access control (like RBAC), enterprise-based access systems (like LDAP), data encryption, secrets management, and secure network connections. | Reduces business risk and protects intellectual property. Instills confidence in end-users. |
| 7) Built in compliance | The solution has capabilities and policies built in to ensure that code being delivered is compliant and secure in the event of an audit. Be able to track and manage important events, trace them back to who performed specific actions, as well as when they occurred.  | Automated security and compliance testing baked into CI pipelines. Automatic notifications for compliance issues. Audit controls and policies in place.  | Reduce risk and discover flaws or potential violations earlier in the development process.  |
| 8) Easy to get started | The steps required to successfully setup CI should be simple and straightforward with minimal barrier to entry. The time and effort required for teams to get started from initial installation,  configuration, onboarding users, and delivering value should be short (days, not weeks). | Supports both on-premise and SaaS implementations. Has robust documentation outlining steps to connect to a repository and get a CI server up and running quickly. Supports configuration-as-code or provides a GUI for initial configurations. Web interface to provision users. | Faster time to value. |
| 9) DevOps tools and integrations | The solution supports strong integrations to both upstream and downstream processes such as project management, source code management and version control, artifact repositories, security scanning, compliance management, continuous delivery, etc. A CI solution with strong DevOps integrations means there's flexibility for users who need or want a balance between native capabilities and integrations. | Integrations with build automation tools (Gradle, Maven etc.), code analysis tools, binary repos, CD tools, IDEs, APIs, third party libraries or extensibility via plugins. | Increases efficiency. Lessens cost and the extra work that comes along with potential migrations. |
| 10) Analytics | Capture and report on pipeline performance metrics and data throughout the build/test processes in order to identify bottlenecks, constraints, and other opportunities for improvement. Identify failing tests, code quality issues, and valuable trends in data indicative of overall application quality. | Build health, pipeline visualization, code coverage, logging, and performance metrics such as deployment frequency. | Increase efficiencies and reduce unnecessary costs. |
| 11) Elastic scalability | The solution supports elastic scaling leveraging well understood, third-party, mechanisms whenever possible. Supports build scalability around existing native per-cloud and per-virtual environment vendor capabilities since they represent already debugged code for a complex problem, and the customers architects/operators are much more likely to be familiar with these mechanisms in their chosen infrastructure/cloud provider. | Supports ephemeral provisioning. Utilizes strong Kubernetes integration at-scale or supports non-Kubernetes scaling options. For example, use AWS Autoscaling Groups in AWS, GCP scaling in GCP, and Azure Autoscale in Azure. | Faster and more efficient. Reduce infrastructure overhead with on-demand resources. Provides flexibility to scale large workloads using popular cloud providers. |

| 12) Artifact and dependency management | The solution allows for easy management of outputs and binaries from the build/test process. Manage repos, application packages, and containers with visibility into their dependencies. | View and download artifacts. Modify, store, and share images. Support for a wide range of common package formats and third-party integrations. Can be used on-prem or in the cloud. | Streamlines your CI/CD workflow and speeds up development. |

The GitLab Solution

<iframe width="960" height="569" src="https://www.youtube.com/embed/ljth1Q5oJoo" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

How GitLab Meets the Market Requirements

| Market Requirements | How GitLab Delivers | GitLab Stage/Category | Demos |
| ------ | ------ | ------ | ------ | ----
| Build automation | Architect CI pipelines with .gitlab-ci.yml files, structure CI processes, and build your app using GitLab Runner as the execution agent. Includes CI services, parent-child pipelines, multi-project pipelines, and merge trains | Verify stage: CI, GitLab Runner, Merge Trains | Build and test automation |
| Test automation | Run various automated tests in your CI pipelines to verify/validate code pre-production. Includes Unit tests, integration testing, browser performance testing, code quality, code coverage, usability testing, and accessibility testing. | Verify stage: CI, Code Quality, Code Testing and Coverage, Web Performance, Usability Testing, Accessibility Testing| Build and test automation |
| Pipeline configuration management | AutoDevOps automatically sets your CI/CD configuration based on pre-configured CI/CD templates. | Verify stage <br> Package stage <br> Secure stage <br> Release stage <br> Configure stage <br> Monitor stage | Pipeline configuration management |
| Visibility and collaboration | See code quality analysis and code coverage details from source code. Get feedback on code changes directly in GitLab with merge requests (MRs) and issues: edit, comment, review, and share in one place. Preview changes in review apps, see commit history, and get automatic alerts on important events. | Verify stage: CI, Code Quality, Code Testing and Coverage <br> Create stage: Code Review, Source Code Management, Design Management <br> Configure stage: ChatOps <br> Release stage: Review Apps | Visibility and collaboration |
| Platform and language support | GitLab is multi-platform (Unix, Windows, OSX, and any other platform that supports Go) and multi-language (Java, PHP, Ruby, C, and any other language). | All | tbd |
| Pipeline security | Security automation and scanning capabilities are built into GitLab's CI pipelines (SAST, DAST, dependency scanning, container scanning). Use Security scanning capabilities with Auto DevOps as well. | Verify stage: CI <br> Secure stage | Shifting Security Left - GitLab DevSecOps Overview |
| Built in compliance | Compliance testing and audit controls are built into GitLab's CI pipelines. | Manage stage: Audit Events, Compliance Management <br> Verify stage: CI <br> Secure stage: License compliance| Manage Compliance with GitLab |
| Easy to get started | GitLab supports config-as-code via .gitlab-ci.yml files and Auto DevOps to predefine configurations or a web UI to get started quickly and easily. | Verify stage: CI | Easy to get started |

| DevOps tools and integrations | Slack, Jira, Docker, Kubernetes, external repos like GitHub, Bitbucket, and any other Git-based repo. GitLab also supports various APIs and third-party libraries for connecting external services and build tools, as well as GDK and Frontend Foundations for community contributors. | All | tbd |
| Analytics | Manage and optimize your software delivery lifecycle with metrics and value stream insight in order to streamline and increase your delivery velocity. Visualize pipelines and report on performance metrics such as memory usage, load testing results, code complexity, and code coverage stastics. | Manage stage: Insights, Value Stream Management <br> Verify stage: CI, Code Quality, Code Testing and Coverage, Web Performance | tbd |
| Elastic scalability | Orchestrate and distribute workloads for parallel builds. Autoscale with GitLab Runner. | Verify stage: CI, GitLab Runner <br> Release stage | tbd |
| Artifact and dependency management | Manage packages, repositories, and containers along with their dependencies in GitLab. View/download artifacts. Edit, store, and share images. | Package stage: Package Registry, Container Registry, Dependency Proxy | tbd |

Top GitLab Features for CI

- Multi-platform: you can execute builds on Unix, Windows, OSX, and any other platform that supports Go.
- Multi-language: build scripts are command line driven and work with Java, PHP, Ruby, C, and any other language.
- Parallel builds: GitLab splits builds over multiple machines for fast execution.
- Autoscaling: you can automatically spin up and down VM's or Kubernetes pods to make sure your builds get processed immediately while minimizing costs.
- Realtime logging: a link in the merge request takes you to the current build log that updates dynamically.
- Versioned tests: a .gitlab-ci.yml file that contains your tests, allowing developers to contribute changes and ensuring every branch gets the tests it needs.
- Flexible Pipelines: define multiple jobs per stage and even trigger other pipelines.
- Build artifacts: upload binaries and other build artifacts to GitLab. Easily browse and download them.
- Test locally: reproduce tests locally using gitlab-runner exec.
- Docker support: use custom Docker images, spin up services as part of testing, build new Docker images, run on Kubernetes.
- Container Registry: built-in container registry to store, share, and use container images.
- we'll add to this list as we continue to build out more lovable features!

Top 3 GitLab Differentiators

| Differentiator | Value | Proof Point | Demos |
|---|---|---|---|
| 1) Leading SCM, Code Review, and CI in one application | You'll be able to streamline code reviews and increase collaboration at proven enterprise scale with GitLab, making development workflows easier to manage and minimizing context switching required between tools in complex DevOps toolchains. Release software faster and outpace the competition with the ability to quickly respond to changes in the market. | Forrester names GitLab among the leaders in Continuous Integration Tools in 2017, Alteryx uses GitLab to have code reviews, source control, CI, and CD all tied together. | Leading SCM, CI and Code Review in one application |
| 2) Uniquely enables rapid innovation | Your organization's speed to market is directly impacted by how fast your development teams can move and adapt to change at an individual |

level. GitLab provides a complete DevOps platform that teams can use to innovate faster without sacrificing quality and enables cross-team collaboration in a central location. Combining powerful features "that just work" and leveraging automation in place of manual work wherever possible helps bring teams together across the entire SDLC regardless of role - product managers, designers, developers, managers, and all roles in between, can work more efficiently and stay involved as a part of a single conversation across the SDLC. | GitLab deploys over 160 times a day and is one of the 30 Highest Velocity Open Source Projects from the CNCF, we're voted as a G2 Crowd Leader 2018 with more than 170 public reviews and a 4.4 rating noting "Powerful team collaboration for managing software development projects," and have over 2,900 active contributors. Forrester's Total Economic Impact (TEI) of GitLab study details the cost savings and business benefits of adopting GitLab. |  Uniquely enabling rapid innovation |
| 3) Shift built in security and compliance "all the way left" | You get security features out-of-the-box and automated security testing with audit controls to facilitate policy compliance. Moving security testing farther left into the SDLC catches potential problems earlier and shortens feedback loops for developers. This means a faster time to market delivering secure, compliant, code and an increase in customer confidence. | Gartner mentions GitLab as a vendor in the Application Monitoring and Protection profile in its 2019 Hype Cycle for Application Security. GitLab positioned in the niche players quadrant of the 2020 Gartner Magic Quadrant for Application Security Testing. Wag! takes advantage of built-in security and faster releases with GitLab, and Glympse makes their audit process easier and remediates security issues faster. |  Built in security and compliance |

## CI Use Case Message House

The message house provides a structure to describe and discuss the value and differentiators for Continuous Integration with GitLab.

## GitLab Runner Messaging and Positioning

The runner message house provides a structure to describe and discuss the value and differentiators for GitLab Runner, the open source project that is used to run your jobs and send the results back to GitLab.

## Customer Facing Slides

<figure class="videocontainer">
<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vRmT3xh0VnNckhZOKRJz1x02tfY90ySaYb48YM55HInYMWa8fmSugK6lknvTChiNWSyYgy4ngK9FK3B/embed?start=false&loop=false&delayms=3000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>
</figure>

## Discovery Questions

The sample discovery questions below are meant to provide a baseline and help you uncover opportunities when speaking with prospects or customers who are not currently using GitLab for CI. See a more complete list of questions, provide feedback, or comment with suggestions for GitLab's CI discovery questions and feel free to contribute!

## Sample Discovery Questions

- CI/CD tool sprawl is one of the most common problems we see. How do you manage the complexities of many different teams using various tools and still meet their needs?
- We've heard that some customers struggle with managing complex pipelines and supporting integrations.  What difficulties do you and/or your team see in these areas?  Do you have a sense of how much time the team is spending?
- Many of our customers have multiple Jenkins administrators. Is this the case with you? What would it mean to your organization if you could free half of those people up to do more than manage pipelines?
- How happy are your teams with the usability and interface of their current CI solution?
- Has there been any discussion to standardize on a single solution for CI since you're already using GitLab for other needs?
- How are you currently supporting CI internally? Do you have a dedicated team or require in-house expertise for guidance, best practices, and fixing issues?
- How often is your day to day or planned work interrupted to fix or maintain your CI tool?
- How much productivity is lost because of delays due to managing your Jenkins environment separately from your source code?
- How much time does your team spend 'babysitting' their pipeline jobs?
- Is your organization investing to improve CI/CD in the short term or long term? Is there a clearly defined strategy or timeline?
  - What's the expectation on your team to support or facilitate this initiative?
  - Are you going to be onboarding additional teams in the next say, 12 months?
- What kind of roadblocks or hurdles does your team encounter when automating builds/tests at scale? How is this affecting your velocity as you scale?
- What is the workflow if a developer wants to create a new pipeline or add a job to an existing pipeline? How much time does that take the developer away from doing "real work?"
- What would be the impact if your team were able to self-service a working pipeline within 1 hour with confidence that all standards and best practices are followed?
- Would you be open to scheduling a follow-up call to discuss more about what GitLab CI can do for you and your team?

Additional Discovery Questions

- Has there been any discussion to standardize on a single solution for CI since you're already using GitLab for other needs?
- What is your strategy around improving CI/CD?
- Would it be valuable to have your CD solution use the same configuration and format as your CI, AND have visibility into the full product pipeline from idea to production?
- Tell me about the difficulties you're having managing complex pipelines and supporting integrations.

Jenkins-specific Discovery Questions

Please see the Jenkins-specific questions  (GitLab internal only)

Industry Analyst Relations (IAR) Plan

- The IAR Handbook page has been updated to reflect our plans for incorporating Use Cases into our analyst conversations.
- For  details specific to each use case, and in respect of our contractual confidentiality agreements with Industry Analyst firms, our engagement plans are available to GitLab team

members in the following protected document: IAR Use Case Profile and Engagement Plan.

For a list of analysts with a current understanding of GitLab's capabilities for this use case, please reach out to Analyst Relations via Slack (analyst-relations) or by submitting an issue and selecting the "AR-Analyst-Validation" template.

## Competitive Comparison

Amongst the many competitors in the DevOps space, Jenkins and CircleCI are the closest competitors offering continuous integration capabilities.

## Proof Points - Customer Recognitions

## Quotes and reviews

## Gartner Peer Insights

Gartner Peer Insights reviews constitute the subjective opinions of individual end users based on their own experiences, and do not represent the views of Gartner or its affiliates. Obvious typos have been amended.

>"We've migrated all our products from several "retired" VCS's to GitLab in only 6 months. - Including CI/CD process adoption - Without [loss] of code - Without frustrated developers - Without enormous costs"
>
> - Application Development Manager, Gartner Peer Review
>
>"Finally, the most amazing thing about GitLab is how well integrated the [GitLab] ecosystem is. It covers almost every step of development nicely, from the VCS, to CI, and deployment."
>
> - Software